TL — TaskList

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.1

Campo	Descrição
Projeto	TL – TaskList
Gerente de Projeto	Vitor Hugo da Cunha
Gerente/Responsável de GCS-DevOps	Vitor Hugo da Cunha
Fábrica de Software / Squad	TL - SoftWare
Repositório principal	https://github.com/vitorhugcunnha/taskList

Histórico de Revisões

Data	Versão	Descrição	Autor
21/09/2026	1.1	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Luiz Camargo

Sumário

Sumário 2
1. Introdução 4
1.1 Finalidade 4
1.2 Escopo 4
1.3 Definições, Acrônimos e Abreviações 4
1.4 Referências 5
1.5 Visão Geral 5
2. Cultura DevOps e Papéis na Gerência de Configuração 5
3. Ferramentas, Ambientes e Infraestrutura 7
3.1 Ferramentas do Pipeline 7
3.2 Ambientes e Infraestrutura 7
3.3 Estratégia de Branching e Versionamento 8
4. Identificação da Configuração 8
4.1 Itens de Configuração 9
4.2 Convenção de Nomenclatura 9
4.3 Estrutura do Repositório 9
5. Integração Contínua (CI) 10
5.1 Etapas do Pipeline de CI 10
5.2 Política de Pull Request e Revisão de Código 11
5.3 Critérios de Qualidade (Quality Gate) 11
6. Entrega e Implantação Contínua (CD) 11
6.1 Etapas do Pipeline de CD 12
6.2 Estratégias de Implantação 12
6.3 Rollback e Recuperação 13
7. Infraestrutura como Código (IaC) 13
7.1 Versionamento de Infraestrutura 13
7.2 Ambientes Efêmeros 14
8. Controle de Configuração e Mudança 14
8.1 Processo de Solicitação de Mudança 14
8.2 Comitê de Controle de Mudança (CCM) 14
9. Baselines e Releases 14
9.1 Geração Automática de Baseline 15
9.2 Notas de Versão (Release Notes) 15
10. Estimativa do Status de Configuração 15
10.1 Monitoramento e Observabilidade 15
10.2 Métricas DORA 15
10.3 Relatórios e Auditorias 16
11. Marcos 16

12. Treinamento e Recursos 16
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 17
14. Quadro-Resumo do Pipeline CI/CD 17

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) aplicadas ao projeto **TaskList**, uma aplicação web para gerenciamento de tarefas desenvolvida com React e TypeScript. O plano contempla práticas de controle de versão, Integração Contínua (CI), Entrega/Implantação Contínua (CD), testes automatizados e automação de pipelines por meio do GitHub Actions. As atividades têm como objetivo identificar e controlar os itens de configuração do projeto, garantir rastreabilidade das alterações e manter a integridade das versões durante o ciclo de desenvolvimento. A utilização de Infraestrutura como Código (IaC) não se aplica ao escopo atual do projeto, pois a aplicação não possui infraestrutura própria provisionada, sendo publicada por meio do serviço gerenciado GitHub Pages.

1.1 Finalidade

Estabelecer um padrão de Gestão de Configuração de Software para o desenvolvimento do **TaskList**, garantindo rastreabilidade, versionamento e controle das mudanças realizadas no produto. O processo é apoiado pelo Git, GitHub, Pull Requests e pipelines automatizados responsáveis pela análise de código, execução de testes, validação do build e implantação da aplicação.

1.2 Escopo

Este Plano de Gerenciamento de Configuração é destinado ao desenvolvimento individual do sistema **TaskList**, sob responsabilidade de **Vitor Hugo da Cunha**. O TaskList é uma aplicação web desenvolvida com React e TypeScript para gerenciamento de tarefas, permitindo criar, editar, excluir, concluir, pesquisar e filtrar tarefas, além de manter os dados armazenados localmente no navegador.

O plano abrange o código-fonte, testes automatizados, arquivos de configuração, dependências, documentação, workflows de CI/CD, controle de branches, Pull Requests, releases e o processo de implantação no GitHub Pages.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação

Termo	Significado
Estratégias de Branchs	Estratégia de versionamento utilizada no projeto com as branches `main`, `dev` e branches temporárias destinadas a funcionalidades, testes, CI/CD e documentação.
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)
Pull Request (PR)	Solicitação de integração de alterações entre branches, permitindo validação das mudanças e execução automática da pipeline de CI antes do merge.
Quality Gate	Conjunto de verificações que devem ser aprovadas antes da integração de uma alteração, incluindo lint, testes automatizados e build da aplicação.

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) - adaptado.
- Repositório do projeto TaskList: https://github.com/vitorhugcunnha/taskList
- Documentação técnica do projeto: arquivo README.md disponível no repositório.
- Pipeline de Integração Contínua: .github/workflows/ci.yml.
- Pipeline de implantação: .github/workflows/deploy.yml.
- Release estável do projeto: TaskList v1.0.0.
- Aplicação publicada no GitHub Pages: https://vitorhugcunnha.github.io/taskList/

1.5 Visão Geral

No projeto **TaskList**, a Gestão de Configuração de Software é integrada ao fluxo de desenvolvimento por meio do Git e do GitHub. As alterações são realizadas em branches específicas e registradas através de commits seguindo o padrão Conventional Commits. Após o desenvolvimento, as mudanças são submetidas por Pull Requests e validadas automaticamente pelo GitHub Actions.

A pipeline de Integração Contínua executa instalação das dependências, análise estática com ESLint, testes automatizados com Vitest e validação do build de produção. Após a integração das alterações na branch main, o workflow de implantação publica automaticamente a aplicação no GitHub Pages.

Os commits, Pull Requests, execuções do GitHub Actions, histórico de deploys, tags e releases constituem as principais evidências de rastreabilidade do projeto. A primeira versão estável foi registrada através da tag e release **v1.0.0**.

2. Cultura DevOps e Papéis na Gerência de Configuração

No projeto **TaskList**, as responsabilidades relacionadas ao desenvolvimento, testes, integração, implantação e gerenciamento de configuração são exercidas pelo próprio desenvolvedor responsável pelo projeto. Por se tratar de um projeto individual, os papéis tradicionais de uma equipe DevOps são acumulados por **Vitor Hugo da Cunha**, com apoio das automações disponibilizadas pelo GitHub Actions.

O processo adotado busca reduzir atividades manuais por meio de versionamento com Git, Pull Requests, testes automatizados, análise estática de código, pipelines de CI/CD e implantação automatizada no GitHub Pages.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Responsável por definir o escopo do TaskList, organizar as funcionalidades, priorizar as alterações e decidir quando uma versão está pronta para integração e publicação.
Engenheiro DevOps / Release Engineer	Define a estratégia de branches e versionamento, mantém os workflows do GitHub Actions, acompanha as execuções de CI/CD e controla tags e releases do projeto.
Desenvolvedor(a)	Implementa funcionalidades, utiliza branches específicas, Conventional Commits e Pull Requests.
QA / Automação de Testes	Mantém e executa os testes automatizados com Vitest e React Testing Library, além de validar lint e build antes da integração das alterações.
Segurança (DevSecOps)	Acompanha o processo de implantação da aplicação no GitHub Pages e verifica se a versão publicada está disponível e funcional após o deploy.
Comitê de Controle de Mudanças (CCM)	Não existe um Comitê de Controle de Mudanças formal. Por se tratar de um projeto individual e acadêmico, as mudanças são avaliadas pelo próprio responsável e seguem o fluxo de branch → commit → Pull Request → pipeline de CI → merge.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento do código-fonte, controle de branches, commits, Pull Requests, tags e releases
Orquestração de CI/CD	GitHub Actions	Automação das etapas de lint, testes, build e implantação
Framework de interface	React	Construção da interface da aplicação
Linguagem	TypeScript	Desenvolvimento com tipagem estática
Build e ambiente de desenvolvimento	Vite	Execução local e geração do build de produção
Testes automatizados	Vitest + React Testing Library	Validação do comportamento dos componentes e funções da aplicação
Análise estática	ESLint	Verificação automática da qualidade e padronização do código
Gestão de dependências	npm + package-lock.json	Instalação e versionamento das dependências utilizadas pelo projeto
Empacotamento	Vite (dist/)	Geração dos arquivos estáticos da aplicação para publicação
Registro/artefato de deploy	GitHub Actions + GitHub Pages	Armazenamento temporário e publicação do artefato gerado pelo build
Implantação	GitHub Pages	Hospedagem da aplicação em ambiente de produção
Monitoramento do pipeline	GitHub Actions	Consulta ao status, histórico e logs das execuções de CI/CD
Persistência da aplicação	Local Storage	Armazenamento local das tarefas no navegador
Infraestrutura como Código	Não se aplica	O projeto não provisiona infraestrutura própria
Orquestração de containers	Não se aplica	O projeto não utiliza containers
Gestão de segredos	Não se aplica	A aplicação não utiliza tokens, senhas ou credenciais próprias
SCA / Análise de dependências	Não implementado nesta versão	Não foi configurada ferramenta específica como Dependabot, Snyk ou OWASP Dependency-Check
Rastreamento de mudanças	GitHub Pull Requests + histórico de commits	Rastreabilidade das alterações e integrações realizadas

3.2 Ambientes e Infraestrutura

O projeto TaskList utiliza uma estrutura simplificada de ambientes, compatível com seu escopo acadêmico e com sua arquitetura de aplicação web estática. Não existe infraestrutura própria provisionada pelo desenvolvedor, pois a publicação em produção é realizada através do GitHub Pages, um serviço gerenciado pelo GitHub. Dessa forma, ferramentas de Infraestrutura como Código não são necessárias nesta versão do projeto.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Desenvolvimento e validação local da aplicação utilizando Vite, Node.js e navegador.	Execução local por meio de npm run dev

Ambiente	Finalidade	Gatilho de promoção
Homologação / Staging	Integração das funcionalidades na branch dev, com execução automática de lint, testes e build pelo GitHub Actions.	Pull Request aprovado e pipeline de CI concluído com sucesso
Produção	Versão estável da aplicação armazenada na branch main e publicada no GitHub Pages.	Merge da branch dev para main, seguido de workflow de deploy

O ambiente de produção é disponibilizado através do GitHub Pages no endereço:

https://vitorhugcunnha.github.io/taskList/

Por se tratar de uma aplicação web estática, o projeto não possui servidor próprio, banco de dados, load balancer, endereço IP dedicado ou infraestrutura de containers. A persistência das tarefas é realizada através do Local Storage do navegador.

3.3 Estratégia de Branching e Versionamento

O projeto TaskList utiliza uma estratégia de branching baseada na separação entre desenvolvimento, integração e produção. As alterações são desenvolvidas em branches específicas e posteriormente integradas por meio de Pull Requests.

As principais branches utilizadas são:

• **main** — representa a versão estável e de produção da aplicação. Alterações integradas nessa branch podem disparar automaticamente o workflow de implantação no GitHub Pages.

• **dev** — utilizada como branch de integração e homologação. As funcionalidades são inicialmente integradas nesta branch e validadas pela pipeline de CI antes de serem promovidas para main.

• **feature/<descrição>** — utilizada para implementação de novas funcionalidades. Exemplos: feature/edit-task, feature/local-storage e feature/search-tasks.

• **test/<descrição>** — utilizada para implementação e configuração de testes automatizados. Exemplo: test/task-manager.

• **ci/<descrição>** — utilizada para configurações relacionadas à Integração e Entrega Contínuas. Exemplos: ci/github-actions e ci/github-pages.

• **docs/<descrição>** — utilizada para alterações na documentação do projeto. Exemplo: docs/readme.

Os commits seguem o padrão **Conventional Commits**, utilizando prefixos como feat, test, style, chore, docs e ci.

O projeto utiliza **Versionamento Semântico (SemVer)** no formato MAJOR.MINOR.PATCH. A primeira versão estável foi registrada com a tag e GitHub Release **v1.0.0**.

Nesta primeira versão não foram utilizadas branches específicas de release/* ou hotfix/*. Caso sejam necessárias futuramente, poderão ser incorporadas ao fluxo de desenvolvimento.

4. Identificação da Configuração

4.1 Itens de Configuração

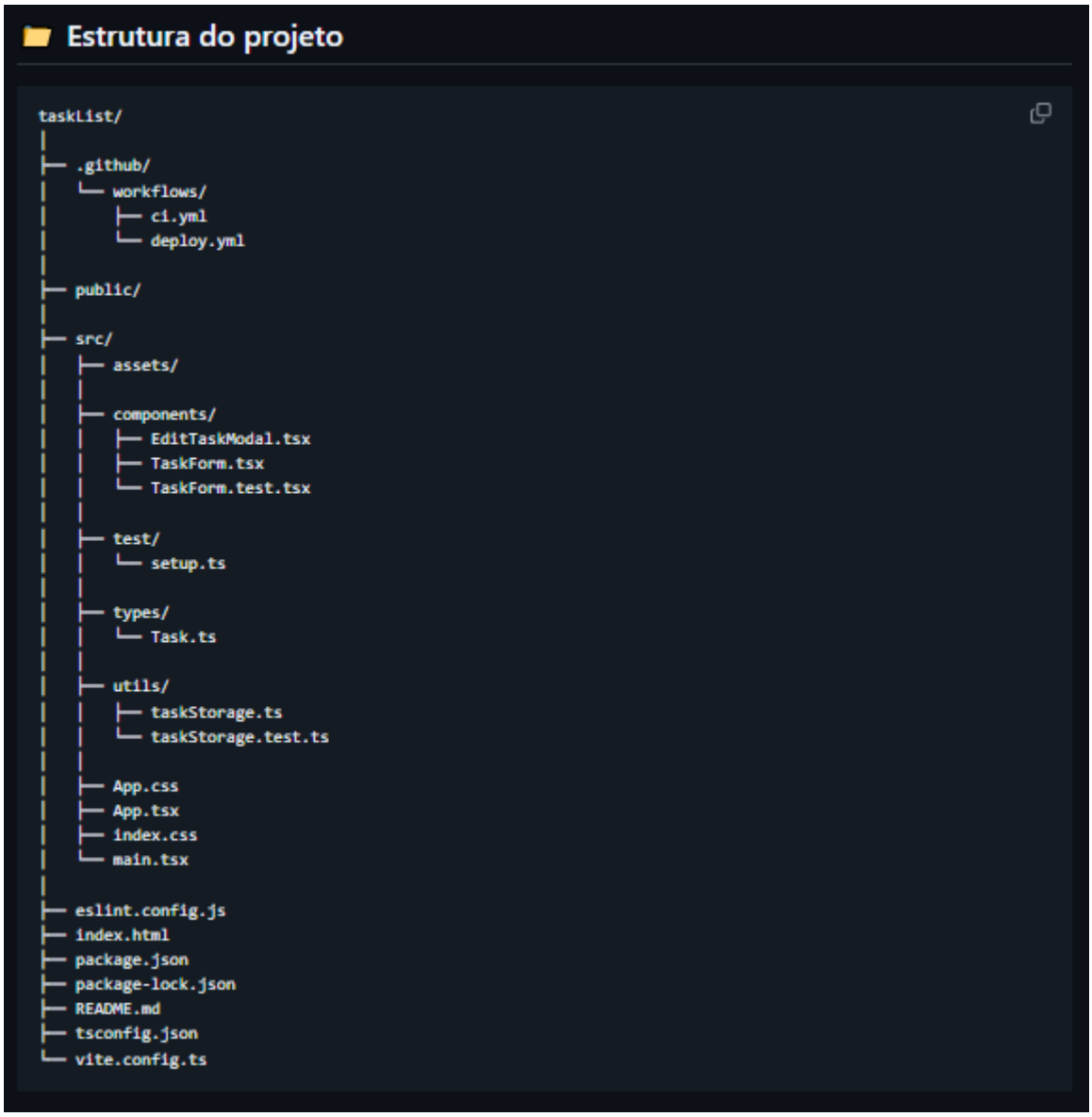
Além do código-fonte, os seguintes artefatos são tratados como itens de configuração formais do projeto TaskList e permanecem versionados no repositório Git:

- Código-fonte da aplicação React/TypeScript;
- Código dos testes automatizados;
- Arquivos de configuração do projeto;
- Arquivos de configuração do Vite;
- Arquivo de configuração do ESLint;
- Arquivos de dependências package.json e package-lock.json;
- Definições de pipeline do GitHub Actions em .github/workflows/;
- Arquivo ci.yml, responsável pela Integração Contínua;
- Arquivo deploy.yml, responsável pela implantação no GitHub Pages;
- Assets e recursos visuais utilizados pela aplicação;
- Documentação técnica do projeto, incluindo o arquivo README.md;
- Tags e releases utilizadas para representar versões estáveis do produto, como v1.0.0.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de funcionalidade	feature/<descrição-curta>	feature/edit-task
Branch de testes	test/<descrição-curta>	test/task-manager
Branch de CI/CD	ci/<descrição-curta>	ci/github-actions
Branch de documentação	docs/<descrição-curta>	docs/readme
Commit	Conventional Commits: <tipo>: <descrição>	feat: add task editing
Commit de testes	test: <descrição>	test: add task form and storage tests
Commit de CI	ci: <descrição>	ci: add GitHub Actions workflow
Commit de documentação	docs: <descrição>	docs: add project documentation
Tag de release	Versionamento Semântico vX.Y.Z	v1.0.0

4.3 Estrutura do Repositório



5. Integração Contínua (CI)

Todo Pull Request direcionado às branches dev ou main dispara automaticamente o pipeline de Integração Contínua configurado no GitHub Actions. O objetivo é validar a qualidade técnica da alteração antes de sua integração, executando análise estática, testes automatizados e build da aplicação.

O pipeline também é executado em pushes realizados nas branches dev e main, permitindo registrar evidências de validação ao longo do processo de desenvolvimento.

5.1 Etapas do Pipeline de CI

1. Checkout do repositório utilizando GitHub Actions.
2. Configuração do ambiente Node.js, utilizando a versão definida no workflow.
3. Instalação das dependências por meio do comando:
npm ci

4. Análise estática do código com ESLint:

npm run lint

5. Execução dos testes automatizados com Vitest e React Testing Library:

npm test

6. Validação do build de produção:

npm run build

7. Conclusão do quality gate, considerando o pipeline aprovado somente quando todas as etapas anteriores forem executadas com sucesso.

5.2 Política de Pull Request e Revisão de Código

• As alterações são desenvolvidas em branches específicas e integradas através de Pull Requests.

• Pull Requests de funcionalidades, testes, CI/CD e documentação são direcionados inicialmente para a branch dev.

• A promoção da versão integrada para produção ocorre por meio de Pull Request da branch dev para main.

• Antes do merge, o Pull Request deve estar livre de conflitos e o pipeline de CI deve ser concluído com sucesso.

• Por se tratar de um projeto individual, não é exigida aprovação de outro desenvolvedor. A validação é realizada pelo próprio responsável do projeto, com apoio do pipeline automatizado de CI.

• Pull Requests devem representar alterações com escopo definido, facilitando rastreabilidade e eventual reversão.

5.3 Critérios de Qualidade (Quality Gate)

Uma alteração é considerada apta para integração somente quando atende aos seguintes critérios:

• Zero erros bloqueantes identificados pelo ESLint;

• Todos os testes automatizados executados pelo Vitest devem ser aprovados;

• O build de produção deve ser concluído com sucesso;

• O workflow Lint, Test and Build do GitHub Actions deve apresentar status aprovado;

• O Pull Request não deve possuir conflitos com a branch de destino.

6. Entrega e Implantação Contínua (CD)

No projeto **TaskList**, a Entrega e Implantação Contínua é realizada por meio do GitHub Actions e do GitHub Pages. Após a integração de uma versão estável na branch main, o workflow de deploy é executado automaticamente, realizando novamente as validações necessárias, gerando o build de produção e publicando a aplicação no GitHub Pages.

O processo permite que a versão presente na branch main seja implantada de forma automatizada no ambiente de produção, mantendo o histórico das execuções e dos deploys no GitHub Actions.

6.1 Etapas do Pipeline de CD

1. **Disparo automático do workflow de deploy** após push ou merge na branch main.
2. **Checkout do código-fonte** correspondente à versão presente na branch main.
3. **Configuração do ambiente Node.js** utilizado pelo projeto.
4. **Instalação das dependências** com:

npm ci

5. **Execução da análise estática** com ESLint:

npm run lint

6. **Execução dos testes automatizados** com Vitest:

npm test

7. **Geração do build de produção**:

npm run build

8. **Configuração do GitHub Pages** para receber a nova versão da aplicação.
9. **Upload do diretório dist/** como artefato de implantação.
10. **Deploy automático do artefato no GitHub Pages**.
11. **Verificação do status da execução** através do GitHub Actions e validação manual da aplicação publicada.

A aplicação em produção pode ser acessada em:

https://vitorhugcunnha.github.io/taskList/

Nesta versão do projeto não foram implementados testes automatizados E2E ou smoke tests após o deploy. A validação pós-implantação é realizada através do status do workflow e da verificação funcional da aplicação publicada.

6.2 Estratégias de Implantação

Estratégia	Descrição
Implantação estática automatizada	Após a integração de alterações na branch main, o GitHub Actions gera o build da aplicação e publica os arquivos estáticos no GitHub Pages.

Estratégia	Descrição
Substituição da versão publicada	A nova versão do diretório dist/ passa a representar a versão disponível em produção após a conclusão bem-sucedida do workflow de deploy.
Validação pós-deploy	O responsável verifica o status do GitHub Actions e acessa a aplicação publicada para confirmar seu funcionamento.

Estratégias como Rolling Update, Blue-Green e Canary Release não se aplicam à arquitetura atual do TaskList, pois a aplicação é composta por arquivos estáticos hospedados no GitHub Pages e não utiliza múltiplas instâncias de servidores ou containers.

6.3 Rollback e Recuperação

O processo de implantação do TaskList é reversível por meio do histórico mantido pelo Git e pelo GitHub. Caso uma versão publicada apresente problemas, o responsável pelo projeto pode identificar a alteração problemática através dos commits, Pull Requests e histórico de deploys.

A recuperação pode ser realizada através da reversão do commit ou merge responsável pelo problema, seguida de uma nova execução do workflow de implantação na branch main.

Também é possível utilizar uma tag de versão estável, como v1.0.0, como referência para identificar o estado anterior do código e restaurá-lo quando necessário.

Como o projeto não utiliza containers ou infraestrutura própria, não existe rollback baseado em imagens Docker ou Infraestrutura como Código. A recuperação é baseada no versionamento do código-fonte e em uma nova implantação pelo GitHub Actions.

7. Infraestrutura como Código (IaC)

A prática de Infraestrutura como Código (IaC) não é aplicada ao escopo atual do projeto **TaskList**. A aplicação é composta por arquivos estáticos gerados pelo Vite e hospedados através do GitHub Pages, serviço gerenciado pelo GitHub.

O projeto não possui servidores próprios, máquinas virtuais, containers, banco de dados ou recursos de infraestrutura que necessitem de provisionamento por ferramentas como Terraform, Ansible ou Pulumi.

Embora os workflows de CI/CD estejam versionados no repositório através dos arquivos presentes em .github/workflows/, esses arquivos representam **pipeline como código**, e não Infraestrutura como Código.

7.1 Versionamento de Infraestrutura

Não se aplica à versão atual do projeto, pois nenhuma infraestrutura própria é provisionada ou administrada pelo responsável pelo TaskList.

As configurações relacionadas à integração e implantação são versionadas no Git juntamente com o código-fonte, principalmente através dos arquivos:

- .github/workflows/ci.yml
- .github/workflows/deploy.yml

Como não existem recursos provisionados por IaC, o projeto também não possui arquivos de estado (*state files*) de infraestrutura.

7.2 Ambientes Efêmeros

Ambientes efêmeros ou *review apps* não foram implementados nesta versão do TaskList.

A validação das alterações é realizada através do ambiente local de desenvolvimento, da branch dev e das verificações automáticas executadas pelo pipeline de CI em Pull Requests.

Caso o projeto evolua e passe a utilizar infraestrutura própria ou ambientes temporários de revisão, ferramentas de IaC poderão ser incorporadas futuramente.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

As mudanças realizadas no projeto **TaskList**, como novas funcionalidades, correções, testes, alterações de documentação ou ajustes nos pipelines, seguem um fluxo controlado e rastreável.

A partir da identificação de uma necessidade de alteração, é criada uma branch específica a partir da branch dev. A implementação é registrada através de commits seguindo o padrão Conventional Commits. Após a conclusão da alteração, a branch é enviada ao GitHub e submetida a um Pull Request direcionado à branch dev.

O Pull Request dispara automaticamente o pipeline de Integração Contínua, responsável por executar lint, testes automatizados e build da aplicação. Após a aprovação das verificações, a alteração pode ser integrada à branch dev.

Quando um conjunto de alterações está pronto para produção, é criado um novo Pull Request de dev para main. Após a validação do pipeline de CI e realização do merge, o workflow de CD executa automaticamente a implantação da aplicação no GitHub Pages.

Dessa forma, commits, branches, Pull Requests e execuções dos pipelines constituem o registro formal e rastreável das mudanças realizadas no projeto.

8.2 Comitê de Controle de Mudança (CCM)

Por se tratar de um projeto acadêmico desenvolvido individualmente, o **TaskList não possui um Comitê de Controle de Mudanças (CCM) formal**. As decisões relacionadas às mudanças são de responsabilidade de **Vitor Hugo da Cunha**, responsável pelo desenvolvimento e gerenciamento de configuração do projeto.

Mudanças rotineiras seguem diretamente o fluxo estabelecido de branch, commit, Pull Request, pipeline de CI e merge.

Alterações consideradas de maior impacto, como mudanças no processo de implantação, estratégia de branches, configurações dos workflows de CI/CD, atualização significativa de dependências ou alterações que possam comprometer a versão publicada em produção, devem ser avaliadas com maior atenção antes de serem integradas à branch main.

Dessa forma, o controle de mudanças é mantido por meio do processo de versionamento e das validações automatizadas, sem necessidade de um comitê formal para o escopo atual do projeto.

9. Baselines e Releases

9.1 Geração de Baseline

No projeto **TaskList**, uma baseline representa uma versão do sistema que atingiu um estado estável e foi disponibilizada em produção.

A primeira baseline formal do projeto corresponde à versão **v1.0.0**, criada após a integração das funcionalidades na branch main, conclusão bem-sucedida dos pipelines de CI/CD e publicação da aplicação no GitHub Pages.

A baseline é identificada através de uma **tag Git utilizando Versionamento Semântico (SemVer)** e de uma **GitHub Release** associada ao estado correspondente do código-fonte.

A criação da tag e da release é realizada manualmente pelo responsável pelo projeto após a validação da versão em produção. Dessa forma, é possível identificar de maneira precisa qual estado do código corresponde a uma determinada versão estável e utilizá-lo como referência em futuras alterações ou recuperações.

9.2 Notas de Versão (Release Notes)

As notas de versão são registradas na área de **Releases do GitHub** e descrevem as principais alterações presentes em cada versão estável do TaskList.

Na release **v1.0.0**, foram documentadas as principais funcionalidades da aplicação, os mecanismos de qualidade utilizados e as práticas DevOps implementadas, incluindo testes automatizados, ESLint, Integração Contínua, implantação automatizada e GitHub Pages.

Os commits seguem o padrão Conventional Commits, contribuindo para a rastreabilidade das mudanças realizadas entre versões. Entretanto, nesta versão do projeto, a geração das notas de release não é automatizada pelo pipeline; as informações são organizadas e publicadas manualmente pelo responsável do projeto.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

No projeto **TaskList**, o monitoramento está concentrado nos recursos disponibilizados pelo GitHub. As execuções dos pipelines de CI/CD podem ser acompanhadas através do GitHub Actions, que mantém histórico, status e logs detalhados de cada execução.

Após a implantação, a disponibilidade da aplicação é verificada através do GitHub Pages e por validação funcional manual da aplicação publicada.

Nesta versão do projeto não foram implementadas ferramentas dedicadas de observabilidade, como Prometheus, Grafana ou serviços de APM, pois o TaskList é uma aplicação web estática de escopo acadêmico e não possui infraestrutura própria de backend.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

As métricas DORA não são coletadas formalmente nesta primeira versão acadêmica do TaskList. Entretanto, o histórico de commits, Pull Requests, execuções do GitHub Actions e deploys no GitHub Pages fornece dados que podem ser utilizados para sua futura apuração.

Como o projeto possui apenas uma primeira release estável (v1.0.0) e não houve histórico suficiente de incidentes em produção, métricas como Change Failure Rate e MTTR ainda não possuem uma amostra significativa para análise.

10.3 Relatórios e Auditorias

O histórico mantido pelo Git e pelo GitHub constitui a principal fonte de auditoria da Gestão de Configuração do TaskList. Os commits permitem identificar as alterações realizadas e seus respectivos autores, enquanto os Pull Requests registram o processo de integração entre as branches.

O GitHub Actions mantém o histórico das execuções de lint, testes, builds e deploys, incluindo seus respectivos logs e resultados. As tags e GitHub Releases permitem identificar as versões consideradas estáveis, como a baseline v1.0.0.

Dessa forma, commits, branches, Pull Requests, pipelines, deploys, tags e releases fornecem evidências rastreáveis do estado da configuração do projeto ao longo de seu desenvolvimento.

11. Marcos

Marco	Descrição	Situação
Criação do projeto	Inicialização da aplicação TaskList utilizando React, TypeScript e Vite	Concluído
Estruturação do versionamento	Criação e utilização das branches main, dev e branches temporárias para funcionalidades e configurações	Concluído
Implementação das funcionalidades principais	Desenvolvimento das funcionalidades de criação, edição, exclusão, conclusão, pesquisa, filtros, categorias e datas	Concluído
Persistência local	Implementação do armazenamento das tarefas utilizando Local Storage	Concluído
Testes automatizados	Implementação de testes utilizando Vitest e React Testing Library	Concluído
Análise estática	Configuração do ESLint para validação do código	Concluído
Pipeline de CI	Implementação do workflow de lint, testes e build utilizando GitHub Actions	Concluído
Pipeline de CD	Implementação do workflow automático de implantação utilizando GitHub Actions e GitHub Pages	Concluído
Primeira implantação em produção	Publicação da aplicação TaskList no GitHub Pages	Concluído

Marco	Descrição	Situação
Documentação técnica	Criação e atualização do arquivo README.md	Concluído
Primeira baseline	Criação da tag e GitHub Release v1.0.0	Concluído

12. Treinamento e Recursos

Por se tratar de um projeto individual, as atividades de desenvolvimento, testes, Gestão de Configuração e DevOps são realizadas por **Vitor Hugo da Cunha**.

Para desenvolvimento e manutenção do projeto são utilizados recursos de software e serviços gratuitos ou disponíveis para fins acadêmicos, sem necessidade de infraestrutura física dedicada.

Recurso / Conhecimento	Utilização no projeto
Visual Studio Code	Ambiente utilizado para desenvolvimento e edição do código
Node.js e npm	Execução do ambiente JavaScript e gerenciamento de dependências
Git	Controle de versão local
GitHub	Hospedagem do repositório, branches, Pull Requests, tags e releases
React	Desenvolvimento da interface da aplicação
TypeScript	Linguagem utilizada no desenvolvimento
Vite	Ambiente de desenvolvimento e geração do build
Vitest	Execução dos testes automatizados
React Testing Library	Testes dos componentes React
ESLint	Análise estática e validação da qualidade do código
GitHub Actions	Automação dos pipelines de CI/CD
GitHub Pages	Hospedagem da aplicação em produção
Git, branches e Pull Requests	Conhecimento necessário para o processo de Gestão de Configuração
YAML e CI/CD	Conhecimento necessário para configuração e manutenção dos workflows

Não são necessários treinamentos específicos em Docker, Kubernetes ou ferramentas de Infraestrutura como Código para a versão atual do projeto, pois essas tecnologias não fazem parte de sua arquitetura. Caso o TaskList evolua para uma aplicação com backend ou infraestrutura própria, novos conhecimentos e recursos poderão ser incorporados ao processo.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

O projeto **TaskList** não utiliza software desenvolvido por subcontratados ou fornecedores externos específicos. Entretanto, utiliza bibliotecas e ferramentas de terceiros disponibilizadas através do ecossistema npm.

As dependências do projeto são declaradas no arquivo package.json e suas versões instaladas são registradas no arquivo package-lock.json, garantindo maior reprodutibilidade entre ambientes e execuções do pipeline.

Entre as principais dependências utilizadas estão React, TypeScript, Vite, Lucide React, Vitest, React Testing Library e ESLint.

A instalação das dependências no ambiente de Integração Contínua e no processo de deploy é realizada através do comando npm ci, que utiliza o package-lock.json como referência para reproduzir as versões registradas no projeto.

Nesta versão do TaskList não foi configurada uma ferramenta específica de análise automática de vulnerabilidades de dependências, como Dependabot, Snyk ou OWASP Dependency-Check. Atualizações de dependências são avaliadas e aplicadas manualmente pelo responsável pelo projeto, sendo posteriormente submetidas ao mesmo processo de versionamento e validação utilizado nas demais alterações.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
Desenvolvimento	Implementação da alteração em uma branch específica, com commits seguindo Conventional Commits.	Necessidade de nova funcionalidade, correção, teste, documentação ou configuração
Pull Request para dev	Submissão da alteração para integração e execução das validações automáticas.	Push da branch e abertura/atualização do Pull Request
CI	Instalação das dependências, execução do ESLint, testes automatizados e build da aplicação.	Pull Request ou push para dev e main
Merge em dev	Integração da alteração após aprovação das verificações automáticas.	Pipeline de CI concluído com sucesso
Gate de Release	Validação do conjunto de mudanças antes da promoção para produção através de Pull Request de dev para main.	Versão integrada e validada na branch dev
Merge em main	Promoção da versão considerada estável para a branch de produção.	CI aprovado e ausência de conflitos no Pull Request
CD — Produção	Instalação de dependências, lint, testes, build, upload do diretório dist/ e deploy automático no GitHub Pages.	Push ou merge na branch main
Validação pós-deploy	Verificação do status do workflow e teste funcional da aplicação publicada.	Conclusão do workflow de deploy
Baseline / Release	Registro de uma versão estável através de tag SemVer e GitHub Release.	Validação da versão publicada em produção

O fluxo acima representa o processo adotado no TaskList desde a implementação de uma alteração até sua disponibilização em produção. O GitHub fornece os principais registros de rastreabilidade por meio do histórico de commits, Pull Requests, execuções do GitHub Actions, deploys, tags e releases.